Scalable URL Shortener
Project Build Plan

HLD + LLD + API contract + data model + implementation roadmap + acceptance criteria

Reference: the supplied "Designing URL Shorteners" notes and screenshots

Purpose
This document is the single source of truth for building the project. Claude should follow the phases in order, preserve the contracts below, add tests at each phase, and avoid prematurely introducing infrastructure that is not required by the current phase.

1. Project vision

Build a production-style URL shortener that creates short, human-readable URLs, redirects them efficiently, and records click analytics asynchronously. The implementation should demonstrate practical backend engineering plus scalable system-design concepts.

Primary differentiators from a basic CRUD project:

- Base62 encoding for compact short codes.
- Ticket/range-based unique ID allocation so multiple API instances can generate IDs safely.
- Redis caching for hot redirect paths.
- Kafka-based asynchronous analytics so redirect latency is not coupled to analytics writes.
- Separate URL storage and analytics storage concerns.
- A React dashboard for basic click visualization.

2. Scope and non-goals

In scope	Out of scope for initial release
Create short URL	Custom aliases
Redirect short URL	Advanced billing/subscriptions
Expiration support	Multi-region active-active deployment
Delete/deactivate URL	Kubernetes production cluster
User ownership	Complex RBAC
Click analytics	ML-based bot detection
Redis cache	Global CDN integration
Kafka event pipeline	Exactly-once global analytics semantics

3. Recommended technology stack

Layer	Technology	Reason

Language	Java 21	Strong backend/interview relevance and modern LTS runtime.
Framework	Spring Boot 3.x	REST APIs, DI, validation, configuration, testing.
Database	PostgreSQL	Strong relational guarantees for mappings and range allocation.
Cache	Redis	Low-latency read cache for redirect traffic.
Messaging	Apache Kafka	Durable asynchronous click-event pipeline.
Frontend	React + Vite	Simple dashboard and fast development.
Charts	Recharts	Lightweight analytics visualization.
Build	Maven	Conventional Spring Boot build lifecycle.
Containerization	Docker + Docker Compose	Repeatable local environment for app + infra.
Testing	JUnit 5 + Mockito + Spring Boot Test + Testcontainers	Unit, integration, and infra-backed tests.
Observability	Spring Boot Actuator + structured logs	Health, readiness, and operational visibility.
Security	Spring Security + JWT + BCrypt	Authentication, access control, stateless API authorization, and secure password hashing.

4. Requirements

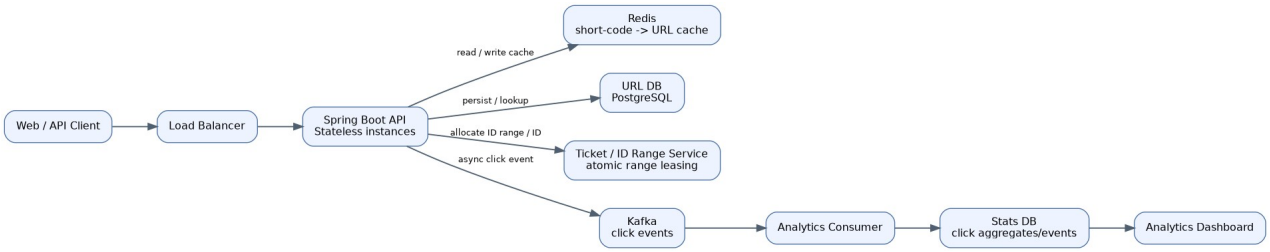
4.1 Functional requirements

- FR-01 Create a short URL from a validated original URL.
- FR-02 Return a short code and fully qualified short URL.
- FR-03 Resolve a short code to the original URL and return HTTP redirect.
- FR-04 Support URL expiration; expired links must not redirect.
- FR-05 Allow link owner to deactivate/delete a short URL.
- FR-06 Record click events with short code, timestamp, user-agent, IP-derived metadata where safely available, device class, browser, and operating system.
- FR-07 Expose aggregate statistics for each owned short URL.
- FR-08 Cache hot mappings in Redis.
- FR-09 Publish click events to Kafka and process them asynchronously.
- FR-10 Provide health/readiness endpoints.
- FR-11 Provide user registration and login using email/username + password.
- FR-12 Issue short-lived JWT access tokens and refresh tokens; support token refresh and logout/revocation.
- FR-13 Associate every managed URL with an authenticated owner_id and enforce owner-only access.
- FR-14 Support basic user profile retrieval and password change.

4.2 Non-functional requirements

- Low redirect latency: cache-first lookup and non-blocking analytics publishing.
- Uniqueness: no two active records may share a short code.
- Horizontal scalability: API instances must be stateless.
- Fault isolation: analytics failure must not normally prevent redirect.
- Observability: health checks, structured logs, and basic metrics.
- Security: validate input, rate limit create endpoints, avoid leaking internal IDs, and never log secrets.

5. High-level architecture (HLD)



Core request paths are deliberately separated: synchronous URL creation/redirects use the URL database and Redis, while analytics flows asynchronously through Kafka to the statistics store.

6. Core design decisions

6.1 Short-code generation

Use a unique numeric ID as the canonical identifier, then encode it using Base62. The numeric ID is not directly exposed to users. The allocation layer leases contiguous ranges of IDs (“tickets”) to API instances or ticket workers, reducing contention on a single global counter.

```
ID allocation example
range = [1_000_000, 1_999_999]
nextId = atomicIncrement(range.current)
shortCode = Base62.encode(nextId)
```

Important
Do not use URL hashing as the primary code-generation strategy. Hashing creates longer codes and makes collision handling part of the hot path. Do not expose sequential numeric IDs directly because they are trivially enumerable.

6.2 Base62 alphabet

```
ALPHABET = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
BASE = 62
```

The encoder must be deterministic and reversible for the numeric ID domain. Unit-test values 0, 1, 61, 62, 63 and representative large IDs.

6.3 Ticket/range allocation

The ticket service owns ranges of numeric IDs. A range row is updated atomically so concurrent requests cannot receive the same ID. When the current value reaches the end, the range is exhausted and the allocator claims another range.

Field	Meaning
range_id	Unique ticket-range identifier.
range_start	Inclusive minimum ID.
range_end	Exclusive maximum ID.
next_id	Next ID available in the range.
status	AVAILABLE, LEASED, EXHAUSTED.
owner_instance	Optional API/ticket worker identifier.
leased_until	Optional lease expiry for abandoned ranges.
owner_id	FK to users.id; required for user-created URLs so ownership is enforced.

6.4 Redis caching strategy

- Key: url:{shortCode}
- Value: original URL + expiration metadata if useful.
- Read-through on redirect: Redis first, PostgreSQL on miss.
- TTL can be aligned with URL expiration where practical.
- On delete/deactivate: evict cache entry.
- Cache failure must fall back to PostgreSQL.

6.5 Kafka analytics strategy

- Redirect path publishes a ClickEvent after the destination URL is known.
- Publishing should be non-blocking from the user's perspective; analytics loss policy must be explicit in implementation.
- Consumer enriches/parses request metadata and updates aggregate statistics.
- Consumer must be idempotent or use a deduplication key if retries can cause duplicate counting.

7. Data model / LLD

7.1 URL table

```
urls
-----
id BIGSERIAL PRIMARY KEY
short_code VARCHAR(16) UNIQUE NOT NULL
original_url TEXT NOT NULL
owner_id BIGINT NULL
created_at TIMESTAMPTZ NOT NULL
expires_at TIMESTAMPTZ NULL
is_active BOOLEAN NOT NULL DEFAULT TRUE
```

Index	Purpose
UNIQUE(short_code)	Fast and safe redirect lookup + collision

	protection.
INDEX(owner_id, created_at DESC)	List a user's links.
INDEX(expires_at)	Optional cleanup/maintenance jobs.

7.2 Ticket range table

ticket_ranges
---
id BIGSERIAL PRIMARY KEY
range_start BIGINT NOT NULL
range_end BIGINT NOT NULL
next_id BIGINT NOT NULL
status VARCHAR(16) NOT NULL
owner_instance VARCHAR(128) NULL
leased_until TIMESTAMPTZ NULL

7.3 Click event / analytics tables

click_events
---
id UUID PRIMARY KEY
short_code VARCHAR(16) NOT NULL
event_time TIMESTAMPTZ NOT NULL
ip_hash VARCHAR(128) NULL
user_agent TEXT NULL
device_type VARCHAR(32) NULL
browser VARCHAR(64) NULL
os VARCHAR(64) NULL
country VARCHAR(64) NULL

daily_url_stats
---
short_code VARCHAR(16) NOT NULL
stat_date DATE NOT NULL
total_clicks BIGINT NOT NULL
mobile_clicks BIGINT NOT NULL
desktop_clicks BIGINT NOT NULL
PRIMARY KEY(short_code, stat_date)

8. LLD - backend components

Component	Responsibility	Key methods
UrlController	REST boundary and validation.	createUrl(), listUrls(), deleteUrl(), getStats()
RedirectController	Resolve code and redirect.	redirect(shortCode)
UrlService	Business logic for creation and ownership.	create(), deactivate(), listByOwner()
RedirectService	Cache-first resolution.	resolveOriginalUrl(), buildRedirect()
ShortCodeService	ID -> Base62 and code rules.	generateCode(), encode(),

		decode()
TicketService	Lease and consume ID ranges.	leaseRange(), nextId(), releaseRange()
UrlRepository	Persistence for URL mappings.	findByShortCode(), save(), findByOwnerId()
TicketRangeRepository	Atomic range allocation.	claimRange(), incrementNextId()
RedisUrlCache	Cache abstraction.	get(), put(), evict()
ClickEventProducer	Publish analytics events.	publish(event)
ClickEventConsumer	Consume and aggregate events.	consume(event)
StatsService	Read analytics aggregates.	getStats(shortCode, ownerId)
GlobalExceptionHandler	Consistent API errors.	handleValidation(), handleNotFound(), handleRateLimit()

9. Backend API contract

Method	Route	Purpose	Auth
POST	/api/v1/urls	Create short URL	Required
GET	/api/v1/urls	List current user's URLs	Required
GET	/api/v1/urls/{shortCode}	Get URL metadata	Required for owner view
DELETE	/api/v1/urls/{shortCode}	Deactivate URL	Required
GET	/api/v1/urls/{shortCode}/stats	Get analytics	Required for owner
GET	/{shortCode}	Public redirect	Public
GET	/actuator/health	Liveness/health	Internal
GET	/actuator/metrics	Metrics	Internal
POST	/api/v1/auth/register	Create user account	Public
POST	/api/v1/auth/login	Authenticate user and issue access/refresh tokens	Public
POST	/api/v1/auth/refresh	Exchange refresh token for a new access token	Public*
POST	/api/v1/auth/logout	Revoke refresh token/session	Required
GET	/api/v1/auth/me	Get current authenticated user	Required
PATCH	/api/v1/auth/me/password	Change current user password	Required

9.1 Create request/response

```
POST /api/v1/urls
Authorization: Bearer <token>
Content-Type: application/json

{
  "originalUrl": "https://example.com/a-long-path",
```

```
  "expiresAt": "2026-12-31T23:59:59Z"
}

201 Created
{
  "shortCode": "a8Kx2",
  "shortUrl": "https://short.example/a8Kx2",
  "originalUrl": "https://example.com/a-long-path",
  "expiresAt": "2026-12-31T23:59:59Z"
}
```

9.2 Redirect semantics

Use HTTP 302 for the initial implementation so behavior is easy to change during development. A later optimization may use 301/308 where link permanence is guaranteed. Return 404 for unknown/deactivated codes and 410 for expired codes if that distinction is intentionally exposed.

9.3 Error contract

```
{
  "timestamp": "2026-08-21T10:30:00Z",
  "status": 400,
  "error": "VALIDATION_ERROR",
  "message": "originalUrl must be a valid HTTP(S) URL",
  "path": "/api/v1/urls",
  "requestId": "..."
}
```

10. Create URL sequence



1. Validate URL scheme and basic format.
2. Authenticate the user and attach owner_id.
3. Request a unique numeric ID from TicketService.
4. Encode the ID with Base62.
5. Persist mapping in PostgreSQL with a unique short_code constraint.
6. Write mapping to Redis when appropriate.
7. Return 201 Created.

11. Redirect + analytics sequence



8. Receive GET /{shortCode}.
9. Check Redis first.
10. If cache miss, query PostgreSQL and optionally populate Redis.
11. Reject missing, inactive, or expired links.
12. Publish ClickEvent to Kafka.
13. Return redirect response immediately without waiting for analytics aggregation.
14. Kafka consumer updates raw/aggregate analytics storage.

12. Security and abuse controls

- Accept only http/https URLs in v1; reject javascript:, data:, file:, and other dangerous schemes.
- Use authentication for create/manage/stats endpoints. Public redirect remains unauthenticated.
- Rate-limit URL creation per user/IP.
- Hash or truncate sensitive IP data instead of storing raw IP where possible.
- Do not expose internal numeric IDs or ticket-range details in API responses.
- Validate request size and reject unreasonable URL lengths.
- Use parameterized JPA/SQL access; never concatenate SQL strings.
- Add security headers and CORS policy appropriate to the frontend deployment.

13. Frontend scope

Screen	Required elements
Create URL	Input, optional expiration, create button, copy short URL.
My URLs	Table of owned short links with status, created date, click count, delete.
Analytics	Selected URL, total clicks, clicks by day, device split, top country/browser.
Error states	Validation, expired link, unauthorized, server unavailable.

14. Repository/package structure

```
url-shortener/
├── backend/
│   ├── src/main/java/com/example/urlshortener/
│   │   ├── controller/
│   │   ├── service/
│   │   ├── repository/
│   │   ├── entity/
│   │   ├── dto/
│   │   ├── cache/
│   │   ├── kafka/
│   │   ├── security/
│   │   ├── config/
│   │   └── exception/
│   └── src/test/java/...
├── frontend/
├── docker-compose.yml
├── docs/
└── README.md
```

15. Implementation phases

Phase	Name	Work	Exit criterion
0	Project skeleton	Repo, Maven, Spring	App starts; DB health

		Boot, Docker Compose, PostgreSQL, environment config.	check passes.
1	Core URL shortener	URL entity, Base62, create + redirect, migrations, validation.	Create and redirect work end-to-end.
2	Ticket ID allocation	Range table, atomic allocation, range leasing abstraction.	Concurrent tests never duplicate IDs.
3	Redis	Cache abstraction, cache-first redirect, eviction, TTL.	Cache HIT bypasses DB; cache outage falls back.
4	Authentication + ownership	Users, password hashing, Spring Security filter chain, JWT access/refresh tokens, logout/revocation, owner_id, authorization rules, password change, and rate limiting for auth endpoints.	Users can register/login/refresh/logout; protected APIs require a valid access token; users cannot manage another user's URLs.
5	Kafka analytics	ClickEvent, producer, topic, consumer, aggregates.	Redirect succeeds even if consumer is offline; events are retried.
6	Stats APIs	Aggregate queries and DTOs.	Stats endpoint returns deterministic results.
7	React dashboard	Create, list, analytics UI.	Full browser workflow works.
8	Hardening	Rate limiting, Actuator, logs, metrics, integration tests.	Acceptance suite passes.
9	Deployment docs	Production topology, config matrix, runbooks.	A new developer can run the project from README.

16. Detailed build order for Claude

Claude should implement exactly in this order and stop after each phase to run the required checks. Do not generate the entire project in one unverified burst.

15. 1. Create repository and baseline Spring Boot app.
16. 2. Add PostgreSQL + Flyway (or Liquibase) migrations.
17. 3. Implement URL entity/repository and create/redirect APIs.
18. 4. Implement Base62 encoder/decoder with tests.
19. 5. Introduce TicketService behind an interface; implement atomic range allocation.
20. 6. Add Redis abstraction and cache-first redirect path.
21. 7. Add authentication, owner isolation, and rate limiting.

22. 8. Add Kafka producer/consumer and ClickEvent schema.
23. 9. Add stats aggregation + read APIs.
24. 10. Build React pages and API client.
25. 11. Add Testcontainers integration suite and concurrency tests.
26. 12. Add Docker Compose for local end-to-end startup.
27. 13. Add documentation: architecture, setup, API examples, trade-offs, failure modes.
28. 14. Run final acceptance checklist and produce a clean README.

17. Testing strategy

Type	Examples	Minimum bar
Unit	Base62, validators, ticket allocation rules	Core logic > 80% coverage
Repository integration	URL lookup, unique constraint, ticket update	Runs against PostgreSQL/Testcontainers
API integration	Create, redirect, delete, stats	Real Spring context + DB
Cache integration	Redis hit/miss/eviction	Real Redis/Testcontainers
Kafka integration	Publish/consume click events	Real Kafka/Testcontainers or equivalent
Concurrency	100-1000 parallel ID requests	Zero duplicate IDs
E2E	Create -> redirect -> analytics	One automated happy-path flow

18. Failure handling and resilience

Failure	Expected behavior
Redis unavailable	Fall back to PostgreSQL; log/metric cache failure.
Kafka unavailable	Do not block redirect indefinitely. Use bounded retry/outbox strategy if strict durability is required.
DB temporarily unavailable	Return controlled 5xx and preserve request ID for troubleshooting.
Ticket range exhausted	Lease another range; if none available, fail creation with a controlled error.
Consumer crashes	Kafka retains events; consumer resumes from committed offset.
Duplicate Kafka delivery	Consumer deduplicates or uses idempotent aggregation.

19. Capacity assumptions (for design discussions)

Reference scale: roughly 100M newly created URLs per month. This is approximately 3.3M creates/day, ~139K creates/hour, and ~2.3K creates/minute on average. Real systems must plan for peak traffic well above the average. Redirect traffic is expected to be substantially more read-heavy than create traffic, which motivates Redis and stateless API scaling.

The numeric ID space should be much larger than the expected volume. 62^6 Base62 combinations are ~56.8B, so a six-character code can support a very large code space. If a future implementation needs more headroom, seven or eight characters can be used without changing the overall model.

20. Observability

- Add requestId/correlationId to logs.
- Metrics: create rate, redirect rate, cache hit ratio, cache latency, DB latency, Kafka publish failures, consumer lag, duplicate-event count.
- Health: DB, Redis, Kafka connectivity/readiness.
- Log structured JSON in production profile; readable console logs locally.
- Never log Authorization headers, passwords, tokens, or full sensitive request metadata.

21. Docker Compose target

```
services:
  api:
    build: ./backend
  postgres:
  redis:
  kafka:
  kafka-ui:      # optional local admin UI
  frontend:
```

The local developer experience should be: one command to start infrastructure, one command to run tests, and one command to start the frontend. Secrets should be supplied through environment variables or local .env files that are gitignored.

22. Definition of done

- [ ] User can create a short URL and receive a short code.
- [ ] Short URL resolves correctly with redirect semantics.
- [ ] No duplicate short codes under concurrency.
- [ ] Redis cache is used on redirect path.
- [ ] Expired/deactivated URLs do not redirect.
- [ ] User ownership is enforced.
- [ ] Click events are emitted to Kafka.
- [ ] Analytics consumer updates statistics.
- [ ] Stats endpoint returns useful aggregates.
- [ ] React dashboard displays the main analytics.
- [ ] Integration tests use real infrastructure where appropriate.
- [ ] Docker Compose can start the complete local stack.
- [ ] README contains setup, API examples, architecture, and troubleshooting.
- [ ] No secrets or generated build artifacts are committed.

23. Interview discussion points

- Why Base62 instead of hashing?
- Why use a ticket/range allocator?
- How do you prevent duplicate IDs across API instances?
- Why is Redis appropriate for the redirect path?
- Why should analytics be asynchronous?
- What happens if Redis fails?
- What happens if Kafka is down?
- How would you shard the URL database?
- How would you handle 10x peak traffic?
- How would you make the analytics consumer idempotent?
- What are the trade-offs of 301 vs 302 vs 308?
- How would you detect abusive URL creation?
- What data should never be stored or logged?

24. Claude execution instructions (copy exactly as the build brief)

Build brief

Implement this project from Phase 0 through Phase 9 and include Phase 4 as a complete user authentication subsystem. Treat the API contracts, auth routes, user/refresh-token schema, ownership rules, JWT/refresh-token strategy, password hashing rules, Base62 rules, ticket-range semantics, Redis cache-first redirect flow, Kafka analytics flow, and definition-of-done checklist as requirements. Prefer small commits/steps. After each phase: run tests, show changed files, explain design decisions, and only then proceed. Do not replace PostgreSQL, Redis, or Kafka with in-memory mocks in the main implementation. Use Testcontainers for integration tests. Do not add custom aliases, payments, Kubernetes, or unrelated features unless explicitly requested. The completed app must support registration, login, refresh, logout, current-user profile, password change, authenticated URL ownership, protected management/statistics APIs, and public redirects.

25. Final deliverables expected from implementation

- Working backend service with documented APIs.
- PostgreSQL migrations and schema.
- Ticket/range allocator and Base62 encoder with tests.
- Redis cache integration.
- Kafka producer/consumer analytics pipeline.
- React analytics dashboard.
- Docker Compose for local stack.
- Comprehensive unit/integration/concurrency tests.
- Architecture diagram and README.
- Example curl commands and sample payloads.

26. Authentication & User Management

Authentication is a first-class subsystem. The public redirect endpoint remains unauthenticated, while all URL creation, management, and analytics endpoints are protected by Spring Security. The recommended approach is stateless JWT access tokens plus refresh tokens stored server-side in hashed form.

26.1 Authentication architecture

- Frontend sends credentials only to /api/v1/auth/register or /api/v1/auth/login over HTTPS.
- Passwords are hashed with BCrypt (or Argon2 if explicitly chosen later); raw passwords are never stored.
- Login returns a short-lived JWT access token and a long-lived refresh token.
- The access token is validated by a Spring Security OncePerRequestFilter or equivalent resource-server JWT filter.
- The JWT subject contains the user identifier; roles/authorities may be included for future admin functionality.
- Refresh tokens are stored as hashes in the database with expiry, revocation state, device/session metadata, and token-family identifiers.
- Logout revokes the refresh-token session/family. Access tokens remain short-lived so their exposure window is limited.
- Protected URL queries always scope by authenticated user id; never trust owner_id supplied by the client.

26.2 Authentication APIs

Method	Route	Purpose	Auth
POST	/api/v1/auth/register	Create account and return sanitized user profile	Public
POST	/api/v1/auth/login	Validate credentials and issue access + refresh tokens	Public
POST	/api/v1/auth/refresh	Rotate refresh token and issue a new access token	Refresh token
POST	/api/v1/auth/logout	Revoke current refresh-token session	Required
GET	/api/v1/auth/me	Return current user profile	Required
PATCH	/api/v1/auth/me/password	Change password; revoke existing sessions as policy requires	Required

26.3 Register contract

```
POST /api/v1/auth/register
Content-Type: application/json

{
  "email": "user@example.com",
```

```
  "password": "StrongPassword123!",
  "displayName": "Laxman"
}

201 Created
{
  "id": "public-user-id",
  "email": "user@example.com",
  "displayName": "Laxman"
}
```

26.4 Login contract

```
POST /api/v1/auth/login
Content-Type: application/json

{
  "email": "user@example.com",
  "password": "StrongPassword123!"
}

200 OK
{
  "accessToken": "<jwt>",
  "refreshToken": "<opaque-token>",
  "tokenType": "Bearer",
  "expiresIn": 900,
  "user": {
    "id": "public-user-id",
    "email": "user@example.com",
    "displayName": "Laxman"
  }
}
```

26.5 Token strategy

- Access token TTL: about 15 minutes for the initial implementation.
- Refresh token TTL: about 7-30 days depending on product preference.
- Use a strong server-side JWT signing secret for local development; prefer asymmetric signing keys (RS256/ES256) for a multi-service production topology.
- Never put passwords, full refresh tokens, or sensitive personal data inside JWT claims.
- Prefer an opaque random refresh token rather than a JWT refresh token. Store only a cryptographic hash of the refresh token.
- Rotate refresh tokens on use and revoke the previous token to reduce replay risk.
- Treat Authorization headers as secrets: never log them.

26.6 User data model

```
users
-----
id BIGSERIAL PRIMARY KEY
email VARCHAR(320) UNIQUE NOT NULL
```

password_hash VARCHAR(255) NOT NULL
display_name VARCHAR(120) NOT NULL
role VARCHAR(32) NOT NULL DEFAULT 'USER'
is_active BOOLEAN NOT NULL DEFAULT TRUE
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL

refresh_tokens
---
id UUID PRIMARY KEY
user_id BIGINT NOT NULL REFERENCES users(id)
token_hash VARCHAR(128) UNIQUE NOT NULL
token_family UUID NOT NULL
issued_at TIMESTAMPTZ NOT NULL
expires_at TIMESTAMPTZ NOT NULL
revoked_at TIMESTAMPTZ NULL
user_agent TEXT NULL
ip_hash VARCHAR(128) NULL

26.7 Required database constraints and indexes

- users.email must be unique and indexed for login lookup.
- refresh_tokens.token_hash must be unique.
- refresh_tokens(user_id, expires_at) should be indexed for cleanup and session management.
- urls.owner_id should reference users.id and have an index together with created_at for dashboard listing.
- Never expose users.id or internal database IDs if public opaque identifiers can be used by the API.

26.8 Spring Security design

- Permit POST /api/v1/auth/register, POST /api/v1/auth/login, POST /api/v1/auth/refresh, GET /{shortCode}, and health endpoints according to the environment policy.
- Require authentication for /api/v1/auth/me, /api/v1/auth/logout, /api/v1/auth/me/password, and all /api/v1/urls/** management routes.
- Configure a stateless SecurityFilterChain; do not use server-side HTTP sessions for the API.
- Add a JWT authentication filter/resource-server configuration that converts the JWT subject into the authenticated principal.
- Use method-level authorization where useful, e.g. @PreAuthorize for future role-based operations.
- Use a global exception handler that maps authentication failures to 401 and authorization failures to 403 without leaking implementation details.

26.9 Ownership rules

- POST /api/v1/urls uses the authenticated principal to set owner_id server-side.
- GET /api/v1/urls returns only rows where owner_id = authenticatedUserId.
- GET /api/v1/urls/{shortCode}, DELETE /api/v1/urls/{shortCode}, and GET /api/v1/urls/{shortCode}/stats must verify ownership before returning or mutating owner-only data.
- The public redirect /{shortCode} does not require authentication because the short URL is intended to be shareable.

- If an authenticated user requests another user's management resource, return 403 or a deliberately non-enumerating 404 based on the selected privacy policy.

26.10 Frontend authentication flow

- Pages: Register, Login, Dashboard/My URLs, Analytics, Profile/Change Password.
- After login, keep access-token handling centralized in an API client/interceptor.
- When an access token expires, call /api/v1/auth/refresh once, rotate tokens, then retry the original request; prevent refresh loops.
- For a browser deployment, prefer secure, HttpOnly, SameSite refresh-token cookies where the backend is configured for cookie-based refresh. If the implementation uses JSON refresh tokens for simplicity, document the trade-off and never use localStorage for long-lived secrets in a production deployment.
- Logout clears client authentication state and revokes the server-side refresh session.

26.11 Authentication security checklist

- Rate-limit register and login endpoints by IP and identifier.
- Use constant-time credential failure behavior and a generic invalid-credentials response.
- Enforce minimum password length and reject obviously unsafe values.
- Never return password_hash, refresh token hashes, or security internals in API responses.
- Use HTTPS in any non-local environment.
- Consider email verification and password reset as future enhancements; they are not required for v1.
- Revoke all active refresh sessions after a password change unless a more granular security policy is deliberately implemented.

26.12 Authentication test plan

- Register succeeds for a new email and rejects duplicate email.
- Login succeeds with correct credentials and rejects wrong credentials with 401.
- Expired/invalid/revoked access tokens cannot access protected endpoints.
- Refresh rotates tokens and rejects reuse of a revoked refresh token.
- Logout revokes the refresh session.
- A user cannot read, delete, or retrieve analytics for another user's URL.
- Public redirects work without an Authorization header.
- Password change invalidates old credentials and follows the chosen session-revocation policy.
- Auth rate limiting triggers after configured thresholds.

27. Authentication interview discussion points

- Why use short-lived JWT access tokens with refresh tokens?
- Why store refresh-token hashes instead of raw refresh tokens?
- What are the trade-offs between HttpOnly cookies and storing tokens in browser storage?
- How does Spring Security establish the authenticated principal?
- How do you prevent one user from accessing another user's URLs?
- What happens to existing sessions after a password change?
- How would you rotate JWT signing keys in production?

- How do you rate-limit login and registration without locking out legitimate users?

28. Updated final deliverables expected from implementation

- Working authentication service with registration, login, refresh, logout, current-user profile, and password change.
- Secure password hashing and Spring Security configuration.
- JWT access-token validation and refresh-token rotation/revocation.
- User/refresh-token migrations and ownership constraints.
- Protected URL CRUD/management/statistics APIs with owner isolation.
- Working backend service with documented APIs.
- PostgreSQL migrations and schema.
- Ticket/range allocator and Base62 encoder with tests.
- Redis cache integration.
- Kafka producer/consumer analytics pipeline.
- React dashboard including authentication screens.
- Docker Compose for local stack.
- Comprehensive unit/integration/concurrency/authentication tests.
- Architecture diagram and README including authentication/security decisions.
- Example curl commands for register, login, refresh, logout, create URL, redirect, and analytics.